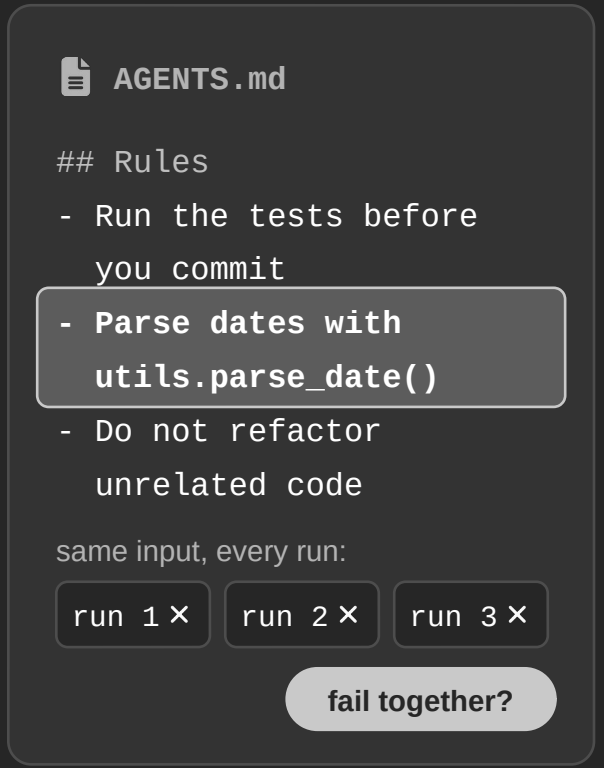
Hiroshima University · Dependable Systems Laboratory · Research plan

Does a shared AGENTS.md make coding-agent runs fail together?

Coincident failures, voting and the reliability of agent-written changes

Sudhanshu Pandit
B.E. CSE, Sri Sairam College of Engineering (VTU), 2026
AI Engineer, founding team, Layerpath
For Assoc. Prof. Junjun Zheng, Dependable Systems Laboratory, Hiroshima University · 2 Nov 2026



About me

2022 to 2026

B.E. Computer Science and Engineering

Sri Sairam College of Engineering (VTU), CGPA 8.0/10
IELTS Academic 8.0

Nov 2025 to now

AI Engineer, founding team, Layerpath

A small startup that makes interactive product demos.
I build its AI features.

From April 2027

Full-time student

I will leave Layerpath before I enrol

What I build

BrandMemory · Layerpath
Gives a company's writing rules to a model as context

Magic Editor · Layerpath
AI assistant that edits product demos; every edit is an action the user approves

Yuki 2.0 · side project, public on GitHub
Desktop agent for Windows, in plain Python with the Anthropic SDK, no LangChain

Every day: I use Claude Code and Codex, review and test their changes, and keep the CLAUDE.md and AGENTS.md files they read

BrandMemory: some rules had no effect, because a 3,200-character limit cut the brand voice section first. After the fix, one A/B comparison: contractions 6 to 0, “you” and “your” 15 to 1.

Background: voting and shared failures

Eckhardt and Lee, 1985 [11]

Two versions drawn at random from one development process fail independently only if every input is equally likely to make a version fail.

Zheng, Okamura and Dohi [1] · **your paper**

Majority voting made strong models, such as GPT-4o, more reliable but helped little with weak ones. More diversity did not always mean higher reliability.

Two 2026 studies: LLM-written versions fail together more often than independence predicts

429

vs ≈ 115

Ron et al. [5]

2 or more of 48 accepted programs (5 agent harnesses) failed together on 429 of one million inputs, vs about 115 expected under independence. Voting over 3 still cut the mean failure count from 387 to 131.

0.43

0.27 if one model

Nogueira et al. [6]

224 contest problems, 12 models. Voting over 3 versions captured 0.43 of the gain that independent failures would allow, and 0.27 when all three came from one model.

The context-file angle: every run in a repository reads the same AGENTS.md or CLAUDE.md, so the file is part of the specification all runs share. If a rule tells every run to use one helper and that helper mishandles an edge case, every run inherits the bug. [5] and [6] used no context file.

Two research questions

The gap, as far as I know: no study has measured how often repeated agent runs on real tasks with a developer's own context file fail together, or whether removing one real rule changes reliability beyond the tests.

1 RQ1 · SHARED FAILURES

When several agent runs solve one real task with the same context file, how much more often do they fail on the same inputs than independence predicts, and which change between runs (file version, agent, model, temperature) lowers it most?

Measure: intraclass correlation ρ of a beta-binomial model (ρ = 0 means independence), and how often a 2-out-of-3 vote fails

2 RQ2 · ONE RULE

Does replacing one real rule with neutral text change the reliability of the code, measured as the failure probability on the pull request's tests and on generated inputs beyond them?

Measure: Original file vs the same file with one rule replaced, on the same tasks; 10,000+ generated inputs per entry point

A checker confirms that each manipulation changed what the agent did. I report compliance but do not test it as an RQ.

Design: repeated runs on real tasks

Real task

Merged PR: its tests fail before, pass after. From the 138 tasks of [8] (12 Python repos), plus newer PRs

Context file

Original, or one rule replaced by neutral text. About 5 rules a script can check, 2 tasks each

Settings, 10 runs each

- Claude Code (CLAUDE.md)
- Codex (AGENTS.md), 2 models
- My harness: Qwen3-30B-Coder (open-weight), 2 temperatures

Version

Admitted only if it passes the PR's tests

Failures beyond the tests RQ2

- 10,000+ random inputs per entry point, from type hints and the PR's test values
- Reference: the merged code; float tolerance, same exception class, order ignored for unordered collections
- 20 disagreements per task checked by hand
- Boundary inputs (BCD method [2]) as a stress test only

Shared failures RQ1

- $\theta(x)$: chance that a run of one setting fails on input x
- Two runs both fail with probability $p^2 + \mathrm{Var}\,\theta(X)$
- Beta model: failing runs out of k are beta-binomial; $\rho = 0$ means independence
- 2-out-of-3 vote fails with probability $E[3\theta^2 - 2\theta^3]$
- Fit by maximum likelihood; bootstrap over runs

820 runs
400 main (10 tasks, 2 file versions, 2 agents, 10 runs), 100 second Codex model, 200 harness, 120 screening

≈ 420 to 590 USD
at the per-task costs in [8]; newer models may cost more

≥ 90%
checker agreement with my hand labels, 20+ examples

Plan, expected results, and why this lab

YEAR 1 · FIRST HALF

Probability, statistics and reliability courses; beta-binomial and NHPP models; harness, checkers, inputs; pilot

YEAR 1 · SECOND HALF

Main runs; failures beyond the tests (RQ2); boundary inputs and mutation checks

YEAR 2 · FIRST HALF

Shared-failure model and voting (RQ1); first paper

YEAR 2 · SECOND HALF

Master's thesis; field data if time permits

Fitted model

How often agent runs fail together, and which change lowers it most: how many runs to vote over at a given cost

One rule

Whether removing one real rule changes reliability beyond the tests

Open data

All tasks, code and data released. Aim: ISSRE or its AISQ workshop, QRS, or Software Quality Journal

Why this lab: your lab's page pairs each technique with “conventional probabilistic evaluation” to show “quantitatively how much the quality actually improved”. RQ1 takes your N-version question [1] to real repository tasks, RQ2 uses the BCD method [2], and field data would use pysrat [4]. My part is the agents and repository mining. I want to learn to fit and test reliability models here.

Thank you. I would be glad to hear your advice.